for you. A chain built with LCEL is itself a runnable, meaning it implements the full `Runnable` interface and can be used like any other component.

COMPOSITION PRIMITIVES

LCEL chains are built by composing existing `Runnables`. The two primary composition primitives are `RunnableSequence` and `RunnableParallel`:

- `RunnableSequence`—Chains `Runnables` sequentially, where the output of one runnable becomes the input for the next:

```
const chain = new RunnableSequence({
first: runnable1,
last: runnable2,
});
```

- `RunnableParallel`—Runs multiple `Runnables` concurrently with the same input. The output is an object with keys corresponding to the `Runnables` and values as their respective outputs:

```
const chain = new RunnableParallel({
   key1: runnable1,
   key2: runnable2,
});
```

- `RunnableLambda`—A special primitive that allows you to define generic JavaScript functions as runnables, making it easy to incorporate custom logic into your chains:

```
const someFunc = RunnableLambda.from((input) => {
return input;
});
const chain = someFunc.pipe(runnable1);
```

The best part is that all of the chains you've already built with runnables are compatible with LCEL, so you can start using it to improve performance, enhance observability, and increase the readability of your code.

6.5.2 *LangGraph*

LangGraph (https://langchain-ai.github.io/langgraphjs/) is an open source library built on top of LangChain that allows you to create stateful, multiactor applications with LLMs. Unlike the linear, chain-based approach of the LCEL, LangGraph uses a graph-based architecture to model workflows with loops and conditional logic. This makes it an ideal tool for building complex, dynamic systems like autonomous agents.

LANGGRAPH VS. LCEL

While LangGraph is built on LangChain's foundation and uses its components, it's designed for a different type of workflow: